

1. What is a System Call?

- **Definition:** A system call is a public function implemented by the OS that acts as the primary interface for user programs to request services from the OS kernel.
- **Core Functionality (Exam Points):**
 1. **Privileged Tasks:** System calls allow programs to do things they aren't allowed to do directly, like communicating with hardware (e.g., reading/writing sectors on a disk).
 2. **Mode Switching:** When a system call is made, the CPU switches from User Mode (restricted) to Kernel Mode (full control). It switches back when the call is done.
 3. **Security and Isolation:** This mechanism prevents buggy or malicious programs from crashing the whole system, keeping critical operations isolated in the kernel.

2. File Interactions System Calls

High-level functions (like cin, cout, or C++ streams) are ultimately built on top of these low-level system calls.

- **open():** Opens a file and returns a File Descriptor (FD). If it fails (e.g., file doesn't exist), it returns -1.
- **close():** Closes an open file descriptor to free up system resources.
- **read():** Reads a specific number of bytes from a file into a memory buffer. Returns the number of bytes read, or 0 if it reaches the End of File (EOF).
- **write():** Writes bytes from a memory buffer into an open file.

3. File Descriptors (FD) vs. Inodes (Crucial Comparison)

(FDs) بالجزء ده (Inodes) دي مقارنة مهمة جداً وموضع أسئلة كثير لأنها بتربط الجزء اللي فات

Feature	File Descriptor (FD)	Inode
Definition	A unique, non-negative integer assigned by the OS representing an <i>open instance</i> of a file within a <i>specific program</i> .	A filesystem-level data structure that represents the <i>actual physical file</i> on the disk.
Scope	Process-specific (Local to your program).	System-wide (Managed by the OS).

Feature	File Descriptor (FD)	Inode
Relationship	Multiple FDs (even in different programs) can point to the <i>same</i> single Inode.	One Inode can be pointed to by multiple FDs.
State Tracking	Each FD maintains its own separate "offset" (current read/write position).	Stores file metadata and block pointers on disk.

💡 How they work together: When you call `read()` or `write()`, you provide the FD. The OS uses the FD to look up your current offset, then uses the corresponding Inode to find the actual physical blocks on the disk.

4. Special File Descriptors & "Everything is a File"

The magic of File Descriptors is that they abstract away the underlying resource. An FD can represent a file, a network connection, or even the terminal!.

Every program starts with 3 special, pre-opened File Descriptors by default:

- 0 (STDIN_FILENO): Standard Input (Reads from the keyboard/terminal). (e.g., `cin` uses FD 0).
- 1 (STDOUT_FILENO): Standard Output (Writes normal output to the terminal).
- 2 (STDERR_FILENO): Standard Error (Writes error messages to the terminal).

📄 Exam Trick / Hack: If you have code that copies data from a `sourceFD` to a `destinationFD` , and you want it to print to the screen instead of saving to a file, you just change the destination FD to 1!

5. Processes and Multiprocessing

- Program vs. Process:
 - Program: The static code you write.
 - Process: An active, running instance of that program (code + execution state). Multiple processes can run the same program simultaneously.
 - PID (Process ID): The OS gives every running process a unique ID. You can get it using the `getpid()` system call.
 - Multiprocessing Illusion: Even on a single-core CPU, the OS switches between processes so fast that it *feels* like they are running simultaneously. The OS is the "scheduler" that decides who runs and when.
-

6. Introducing fork() (The Tricky Part)

- What it does: The fork() system call creates a second, identical process (the Child) which is an exact clone of the original process (the Parent).
- How it executes:
 - The child starts executing from the *exact next instruction* after the fork() call.
 - The parent also continues executing the next instruction.
 - Crucial Exam Note: The OS decides the execution order between the parent and child after the fork; it is non-deterministic.
- The Golden Rule of Fork: fork() is called ONCE, but returns TWICE! (Once in the parent, and once in the newly created child).

وده غالباً هيكون بداية المحاضرة اللي "How do I know if I'm the parent or the child?" :المحاضرة قفلت بسؤال مهم (بترجع قيمة مختلفة في كل واحد فيهم عشان تعرف تفرق fork() بالمذاكرة Hint جاية. بس كـ